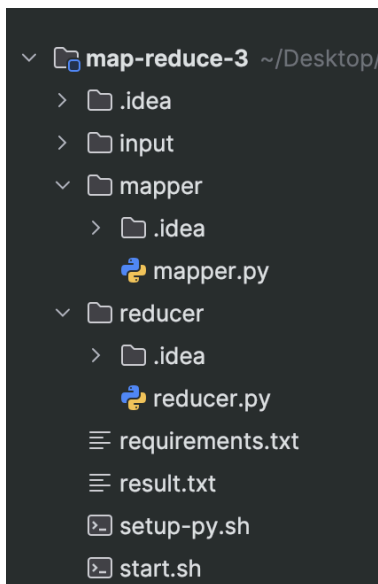


Utilizând algoritmul Map-Reduce (implementarea cu stdin/stdout), să se calculeze indexul invers pentru un set de URL-uri. Funcția de mapare va parsa conținutul fiecărei pagini web (se pot folosi modulele requests și BeautifulSoup) și va returna perechi de forma <word, {URL: 1}>, unde valoarea 1 reprezintă o apariție a cuvântului pe pagina respectivă. Funcția de reducere primește toate perechile, sortează perechile corespunzătoare aceluiași cuvânt și emite o pereche de forma <word, {URL_i: count_word_in_URL_i, ...}>.



NOTITE

1. Prima data instalezi pachetele din stanga jos: requests si beautifulsoup4
2. Creezi fisierele de input si result.txt
3. Faci chmod +x [start.py](#)
4. Rulezi ./start.py

[mapper.py](#)

```
import string

from bs4 import BeautifulSoup
from pyspark.resource import requests

def main():
    urls = [

        "https://ro.wikipedia.org/wiki/Pandemia_de_COVID-19_%C3%AEEn_Rom%C3%A2nia",
        "https://en.wikipedia.org/wiki/Microservices",
        "http://mike.tuiasi.ro/",
        "https://microservices.io/patterns/microservices.html",
```

```

        "https://dzone.com/articles/design-patterns-for-microservices-1",
        "https://docs.python.org/3.3/library/stdtypes.html?highlight=maketrans#str.make
        trans"
    ]
    for url in urls:
        page_content = requests.get(url).content
        soup = BeautifulSoup(page_content, "lxml")
        for tag in soup.find_all("div"):
            table = str.maketrans("", "", string.punctuation)
            words = list(map(lambda s: s.strip().translate(table),
tag.text.split(" ")))
            for word in words:
                if word not in string.punctuation:
                    print("{} {} 1".format(word, url))

if __name__ == "__main__":
    main()

```

[reducer.py](#)

```

import sys

def main():
    dict_result = {}
    fp = open("result.txt", "w")
    for line in sys.stdin:
        line = line.strip("\n")
        data = line.split(" ")
        if data[0] == "":
            continue
        if len(data) != 3:
            continue
        if data[0] in dict_result:
            aux = dict_result[data[0]]
            if data[1] in aux:
                dict_result[data[0]][data[1]] += int(data[2])
            else:
                dict_result[data[0]][data[1]] = int(data[2])
        else:
            dict_result[data[0]] = {data[1]: int(data[2])}
    for key in dict_result:
        fp.write("<{}, {}>\n".format(key, dict_result[key]))
    fp.close()

if __name__ == "__main__":
    main()

```

[start.sh](#)

```
python3 mapper/mapper.py 4300-0.txt 5000-8.txt | python3 reducer/reducer.py
```